

Class and Object in Python (Detailed Explanation)

Python is an **Object-Oriented Programming (OOP)** language. The two most important concepts in OOP are:

1. **Class** → Blueprint or template
 2. **Object** → Real instance created from a class
-

1. What is a Class?

A **class** is a blueprint for creating objects. It defines:

- Attributes (variables/data)
- Methods (functions/behaviors)

Syntax

```
class ClassName:  
    pass
```

Example

```
class Student:  
    name = "John"
```

Here:

- `Student` is a class.
 - `name` is a class attribute.
-

2. What is an Object?

An **object** is an instance of a class.

Example

```
class Student:  
    name = "John"  
  
s1 = Student()  
print(s1.name)
```

Output:

John

Here:

- `Student()` creates an object.
- `s1` is an object of class `Student`.

Real-Life Example

Think of a class as a **car design**.

Class (Blueprint)

```
Car
├── Color
├── Brand
└── Speed()
```

Objects (Actual Cars)

```
Car1 → Red BMW
Car2 → Black Audi
Car3 → White Tesla
```

3. Constructor (`__init__()`)

A constructor is a special method that automatically runs when an object is created.

Syntax

```
class Student:
    def __init__(self):
        print("Object Created")
```

Example

```
class Student:
    def __init__(self):
        print("Student object created")

s1 = Student()
```

Output

Student object created

4. The `self` Keyword

`self` refers to the current object.

Example

```
class Student:
    def __init__(self, name):
        self.name = name

s1 = Student("Alice")
print(s1.name)
```

Output

Alice

Here:

- `self.name` belongs to the object.
- Each object can have different values.

5. Instance Variables

Variables that belong to each object.

Example

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

s1 = Student("Alice", 20)
s2 = Student("Bob", 22)

print(s1.name)
print(s2.name)
```

Output

Alice
Bob

6. Methods in a Class

Methods define object behavior.

Example

```
class Student:
    def __init__(self, name):
        self.name = name

    def display(self):
        print("Name:", self.name)

s1 = Student("Alice")
s1.display()
```

Output

```
Name: Alice
```

7. Multiple Objects

One class can create many objects.

```
class Employee:
    def __init__(self, name):
        self.name = name

e1 = Employee("John")
e2 = Employee("David")
e3 = Employee("Emma")

print(e1.name)
print(e2.name)
print(e3.name)
```

Output

```
John
David
Emma
```

8. Class Variables

Shared by all objects.

Example

```
class Student:
    school = "ABC School"

    def __init__(self, name):
        self.name = name

s1 = Student("Alice")
```

```
s2 = Student("Bob")

print(s1.school)
print(s2.school)
```

Output

```
ABC School
ABC School
```

Difference

Instance Variable	Class Variable
Belongs to object	Belongs to class
Different for each object	Same for all objects
self.name	ClassName.variable

9. Object Methods

Accessing Attributes

```
print(s1.name)
```

Modifying Attributes

```
s1.name = "David"
```

Deleting Attributes

```
del s1.name
```

10. Example Program

```
class Car:
    def __init__(self, brand, color):
        self.brand = brand
        self.color = color

    def start(self):
        print(self.brand, "is starting")

    def show(self):
        print("Brand:", self.brand)
        print("Color:", self.color)

car1 = Car("Tesla", "Red")
car2 = Car("BMW", "Black")
```

```
car1.show()
car1.start()

car2.show()
car2.start()
```

Output

```
Brand: Tesla
Color: Red
Tesla is starting

Brand: BMW
Color: Black
BMW is starting
```

11. Class vs Object

Class	Object
Blueprint	Instance
Logical entity	Physical entity
No memory allocated until object created	Memory allocated
Defines properties and methods	Uses properties and methods
Example: Car	Example: Tesla Car

12. Benefits of Classes and Objects

1. Code Reusability

```
class Student:
    pass
```

Create unlimited student objects.

2. Data Security

Data and methods stay together.

3. Easy Maintenance

Changes in one class affect all objects.

4. Real-World Modeling

Represents real-world entities like:

- Student
 - Employee
 - Bank Account
 - Car
 - Mobile Phone
-

Complete Example

```
class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount

    def withdraw(self, amount):
        self.balance -= amount

    def display(self):
        print("Owner:", self.owner)
        print("Balance:", self.balance)

acc1 = BankAccount("Alice", 1000)

acc1.deposit(500)
acc1.withdraw(200)
acc1.display()
```

Output

```
Owner: Alice
Balance: 1300
```

Summary

- **Class** = Blueprint/template.
- **Object** = Instance of a class.
- `__init__()` = Constructor.
- `self` = Current object reference.
- **Instance variables** belong to objects.
- **Class variables** are shared among all objects.
- **Methods** define object behavior.

A simple formula to remember:

Class → Creates Objects → Objects contain Data (attributes) + Behavior (methods).

Link : CHATGPT

Inheritance in OOP (Python) – Detailed Explanation

Inheritance is one of the most important concepts of **Object-Oriented Programming (OOP)**. It allows one class to inherit the properties and methods of another class.

What is Inheritance?

Inheritance is a mechanism in which one class acquires the attributes and methods of another class.

- The existing class is called the **Parent Class** (Base Class or Super Class).
- The new class is called the **Child Class** (Derived Class or Sub Class).

Syntax

```
class ParentClass:
    # Parent class code

class ChildClass(ParentClass):
    # Child class code
```

Why Use Inheritance?

1. Code Reusability

You can reuse the code of an existing class.

2. Easy Maintenance

Changes made in the parent class are automatically available to child classes.

3. Extensibility

Child classes can add new features.

4. Real-World Modeling

Represents "is-a" relationships.

Example:

```
Dog is an Animal
Car is a Vehicle
Teacher is a Person
```

Basic Example

```
class Animal:
    def eat(self):
        print("Animal is eating")

class Dog(Animal):
    def bark(self):
        print("Dog is barking")

d = Dog()

d.eat()
d.bark()
```

Output

```
Animal is eating
Dog is barking
```

Explanation

```
Animal
  ↓
Dog
```

- Dog inherits from Animal.
- Therefore, Dog can use eat().

Types of Inheritance in Python

Python supports five types of inheritance.

1. Single Inheritance
2. Multiple Inheritance
3. Multilevel Inheritance
4. Hierarchical Inheritance
5. Hybrid Inheritance

1. Single Inheritance

One child class inherits from one parent class.

Example

```
class Father:
    def show(self):
        print("Father's method")

class Son(Father):
    def display(self):
        print("Son's method")

s = Son()

s.show()
s.display()
```

Output

```
Father's method
Son's method
```

Diagram

```
Father
  ↓
Son
```

2. Multiple Inheritance

One child class inherits from multiple parent classes.

Example

```
class Father:
    def father_method(self):
        print("Father Method")

class Mother:
    def mother_method(self):
        print("Mother Method")

class Child(Father, Mother):
    def child_method(self):
        print("Child Method")

c = Child()

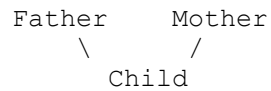
c.father_method()
c.mother_method()
c.child_method()
```

Output

```
Father Method
```

Mother Method
Child Method

Diagram



3. Multilevel Inheritance

A class inherits from a child class.

Example

```
class Grandfather:
    def grand_method(self):
        print("Grandfather Method")

class Father(Grandfather):
    def father_method(self):
        print("Father Method")

class Son(Father):
    def son_method(self):
        print("Son Method")

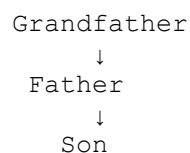
s = Son()

s.grand_method()
s.father_method()
s.son_method()
```

Output

Grandfather Method
Father Method
Son Method

Diagram



4. Hierarchical Inheritance

Multiple child classes inherit from one parent class.

Example

```
class Animal:
    def eat(self):
        print("Animal eats food")

class Dog(Animal):
    def bark(self):
        print("Dog barks")

class Cat(Animal):
    def meow(self):
        print("Cat meows")

d = Dog()
c = Cat()

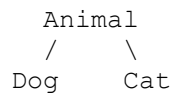
d.eat()
d.bark()

c.eat()
c.meow()
```

Output

```
Animal eats food
Dog barks
Animal eats food
Cat meows
```

Diagram



5. Hybrid Inheritance

Combination of two or more types of inheritance.

Example

```
class A:
    def method_a(self):
        print("Class A")

class B(A):
    def method_b(self):
        print("Class B")

class C(A):
    def method_c(self):
        print("Class C")
```

```
class D(B, C):
    def method_d(self):
        print("Class D")

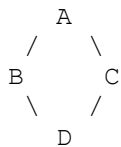
d = D()

d.method_a()
d.method_b()
d.method_c()
d.method_d()
```

Output

```
Class A
Class B
Class C
Class D
```

Diagram



The `super()` Function

`super()` is used to call methods or constructors of the parent class.

Without `super()`

```
class Parent:
    def __init__(self):
        print("Parent Constructor")

class Child(Parent):
    def __init__(self):
        Parent.__init__(self)
        print("Child Constructor")

c = Child()
```

Output

```
Parent Constructor
Child Constructor
```

With `super()`

```
class Parent:
    def __init__(self):
        print("Parent Constructor")

class Child(Parent):
    def __init__(self):
        super().__init__()
        print("Child Constructor")

c = Child()
```

Output

```
Parent Constructor
Child Constructor
```

Constructor Inheritance

Parent constructors are not automatically executed if the child defines its own constructor.

Example

```
class Parent:
    def __init__(self):
        print("Parent Constructor")

class Child(Parent):
    def __init__(self):
        print("Child Constructor")

c = Child()
```

Output

```
Child Constructor
```

Use `super()` if you want both constructors.

Method Overriding

Method overriding means redefining a parent method in the child class.

Example

```
class Animal:
    def sound(self):
        print("Animal makes sound")
```

```
class Dog(Animal):
    def sound(self):
        print("Dog barks")

d = Dog()
d.sound()
```

Output

Dog barks

Method Resolution Order (MRO)

MRO determines the order in which Python searches for methods.

Example

```
class A:
    pass

class B(A):
    pass

class C(A):
    pass

class D(B, C):
    pass

print(D.mro())
```

Output

```
[<class '__main__.D'>,
 <class '__main__.B'>,
 <class '__main__.C'>,
 <class '__main__.A'>,
 <class 'object'>]
```

You can also use:

```
print(D.__mro__)
```

Advantages of Inheritance

- Code Reusability
- Reduces Code Duplication
- Improves Maintainability
- Supports Extensibility
- Represents Real-World Relationships

- Easier Development
-

Disadvantages of Inheritance

- Tight coupling between classes.
 - Changes in the parent class can affect child classes.
 - Deep inheritance hierarchies can become complex.
 - Incorrect use can reduce flexibility.
-

Real-Life Example: Employee System

```
class Employee:
    def __init__(self, name):
        self.name = name

    def show(self):
        print("Employee:", self.name)

class Manager(Employee):
    def manage(self):
        print(self.name, "manages the team")

class Developer(Employee):
    def code(self):
        print(self.name, "writes code")

m = Manager("John")
d = Developer("Alice")

m.show()
m.manage()

d.show()
d.code()
```

Output

```
Employee: John
John manages the team
Employee: Alice
Alice writes code
```

Difference Between Class and Inheritance

Feature	Class	Inheritance
Meaning	Blueprint for objects	Acquiring properties from another class
Purpose	Create objects	Reuse and extend code
Relationship	Defines an entity	Defines parent-child relationship
Example	<code>class Student:</code>	<code>class Dog(Animal):</code>

Summary

- **Inheritance** allows one class to acquire the properties and methods of another class.
- Parent Class = Base/Super Class.
- Child Class = Derived/Sub Class.
- Python supports **5 types of inheritance**.
- `super()` is used to access parent methods and constructors.
- **Method Overriding** allows child classes to provide their own implementation.
- **MRO (Method Resolution Order)** determines how Python searches for methods in multiple inheritance.

Formula to Remember

Inheritance = Code Reusability + Parent-Child Relationship + Extensibility

Link : [CHATGPT](#)

Polymorphism in OOP (Python) – Detailed Explanation

Polymorphism is one of the four fundamental principles of **Object-Oriented Programming (OOP)**.

The four pillars of OOP are:

1. Encapsulation
2. Inheritance
3. Polymorphism
4. Abstraction

What is Polymorphism?

The word **Polymorphism** comes from two Greek words:

- **Poly** = Many
- **Morph** = Forms

So, **Polymorphism** means "many forms."

In programming:

Polymorphism allows the same method, function, or operator to behave differently depending on the object or data it is used with.

Real-Life Example

Think about the word "**Drive**".

Person → Drive a Car
Pilot → Drive an Airplane
Captain → Drive a Ship

The action is the same (**Drive**), but its behavior changes.

This is Polymorphism.

Why Use Polymorphism?

- Improves code reusability.
 - Makes programs flexible.
 - Reduces complexity.
 - Allows writing generic code.
 - Makes maintenance easier.
-

Types of Polymorphism in Python

Python supports two main types of polymorphism:

1. Compile-Time Polymorphism
 2. Run-Time Polymorphism
-

1. Compile-Time Polymorphism

Compile-time polymorphism is achieved using **Method Overloading**.

However, Python does **not support traditional method overloading** like Java or C++.

Traditional Method Overloading (Not Supported)

```
class Calculator:
    def add(self, a, b):
        return a + b

    def add(self, a, b, c):
        return a + b + c
```

Python keeps only the last method.

Example

```
class Test:
    def show(self):
        print("First")

    def show(self):
        print("Second")

t = Test()
t.show()
```

Output

Second

Achieving Overloading in Python

We use default arguments or variable-length arguments.

Example: Default Arguments

```
class Calculator:
    def add(self, a, b, c=0):
        return a + b + c

calc = Calculator()

print(calc.add(2, 3))
print(calc.add(2, 3, 4))
```

Output

5
9

Example: Variable-Length Arguments

```
class Calculator:
    def add(self, *numbers):
        return sum(numbers)

calc = Calculator()

print(calc.add(2, 3))
print(calc.add(2, 3, 4))
print(calc.add(2, 3, 4, 5))
```

Output

5
9
14

2. Run-Time Polymorphism

Run-time polymorphism is achieved using **Method Overriding**.

A child class provides its own implementation of a parent class method.

Example

```
class Animal:
    def sound(self):
        print("Animal makes sound")

class Dog(Animal):
    def sound(self):
        print("Dog barks")

class Cat(Animal):
    def sound(self):
        print("Cat meows")

dog = Dog()
cat = Cat()

dog.sound()
cat.sound()
```

Output

```
Dog barks
Cat meows
```

Polymorphism Using Inheritance

```
class Bird:
    def fly(self):
        print("Bird can fly")

class Sparrow(Bird):
    def fly(self):
        print("Sparrow flies")

class Eagle(Bird):
    def fly(self):
        print("Eagle flies high")

birds = [Sparrow(), Eagle()]

for bird in birds:
    bird.fly()
```

Output

```
Sparrow flies
Eagle flies high
```

Duck Typing in Python

Python follows the concept of **Duck Typing**.

Rule

"If it walks like a duck and quacks like a duck, then it is a duck."

Python focuses on behavior rather than type.

Example

```
class Dog:
    def speak(self):
        print("Woof")

class Cat:
    def speak(self):
        print("Meow")

def make_sound(animal):
    animal.speak()

make_sound(Dog())
make_sound(Cat())
```

Output

```
Woof
Meow
```

Explanation

The function does not care whether the object is a Dog or Cat.

It only requires a `speak()` method.

Operator Overloading

Operators can behave differently depending on operands.

This is also polymorphism.

Example

```
print(5 + 3)
print("Hello " + "World")
print([1, 2] + [3, 4])
```

Output

```
8
Hello World
[1, 2, 3, 4]
```

The + operator:

- Adds integers
- Concatenates strings
- Merges lists

Custom Operator Overloading

Python uses special methods called **Magic Methods**.

Example:

```
__add__ ()
__sub__ ()
__mul__ ()
__eq__ ()
```

Example

```
class Number:
    def __init__(self, value):
        self.value = value

    def __add__(self, other):
        return Number(self.value + other.value)

    def __str__(self):
        return str(self.value)

n1 = Number(10)
n2 = Number(20)

result = n1 + n2

print(result)
```

Output

Built-in Function Polymorphism

Many built-in functions work with different data types.

Example

```
print(len("Python"))
print(len([1, 2, 3, 4]))
print(len((10, 20, 30)))
print(len({1, 2, 3}))
```

Output

```
6
4
3
3
```

The `len()` function behaves differently for each object.

Function Polymorphism

A single function can work with multiple object types.

Example

```
def add(a, b):
    return a + b

print(add(5, 3))
print(add("Hello ", "Python"))
print(add([1, 2], [3, 4]))
```

Output

```
8
Hello Python
[1, 2, 3, 4]
```

Polymorphism with Abstract Classes

```
from abc import ABC, abstractmethod

class Shape(ABC):

    @abstractmethod
    def area(self):
        pass

class Circle(Shape):
    def area(self):
        print("Area of Circle")

class Rectangle(Shape):
    def area(self):
        print("Area of Rectangle")

shapes = [Circle(), Rectangle()]

for shape in shapes:
    shape.area()
```

Output

```
Area of Circle
Area of Rectangle
```

Advantages of Polymorphism

1. Code Reusability

The same interface works for different objects.

2. Flexibility

Easy to extend the program.

3. Reduced Complexity

Less conditional logic is required.

4. Easy Maintenance

Changes can be made without affecting other code.

5. Better Readability

Cleaner and more organized code.

Disadvantages of Polymorphism

- Can make debugging more difficult.
 - Understanding overridden methods may take time.
 - Excessive use can reduce code clarity.
-

Real-Life Example: Payment System

```
class Payment:
    def pay(self, amount):
        pass

class CreditCard(Payment):
    def pay(self, amount):
        print("Paid", amount, "using Credit Card")

class UPI(Payment):
    def pay(self, amount):
        print("Paid", amount, "using UPI")

class Cash(Payment):
    def pay(self, amount):
        print("Paid", amount, "using Cash")

payments = [CreditCard(), UPI(), Cash()]

for payment in payments:
    payment.pay(1000)
```

Output

```
Paid 1000 using Credit Card
Paid 1000 using UPI
Paid 1000 using Cash
```

Polymorphism vs Inheritance

Feature	Polymorphism	Inheritance
Meaning	One interface, many forms	Acquiring properties from another class
Purpose	Different behaviors	Code reuse
Achieved Through	Overriding, overloading, duck typing	Parent-child relationship
Focus	Behavior	Structure

Polymorphism vs Encapsulation

Feature	Polymorphism	Encapsulation
Focus	Many behaviors	Data protection
Purpose	Flexibility	Security
Achieved Through	Overriding, operator overloading	Private members, properties
Example	<code>sound()</code> behaves differently	<code>__balance</code> hides data

Summary

- **Polymorphism** means "**many forms.**"
- The same interface can perform different actions.
- Python supports:
 - Method Overriding (Run-Time Polymorphism)
 - Simulated Method Overloading
 - Duck Typing
 - Operator Overloading
 - Built-in Function Polymorphism
- It improves flexibility, reusability, and maintainability.

Formula to Remember

Polymorphism = One Interface + Multiple Behaviors

Easy Memory Trick

Inheritance → Reuse Code

Encapsulation → Protect Data

Polymorphism → Same Action, Different Results

Abstraction → Hide Complexity

Link : CHATGPT

Encapsulation in OOP (Python) – Detailed Explanation

Encapsulation is one of the four main principles of **Object-Oriented Programming (OOP)**.

The four pillars of OOP are:

1. Encapsulation
 2. Inheritance
 3. Polymorphism
 4. Abstraction
-

What is Encapsulation?

Encapsulation means **wrapping data (variables) and methods (functions) together into a single unit (class) and restricting direct access to some of the object's data.**

In simple words:

Encapsulation is the process of hiding the internal details of an object and allowing access only through controlled methods.

Real-Life Example

Think of an **ATM Machine**.

```
ATM Machine
↓
You enter PIN
↓
ATM verifies PIN
↓
Money is withdrawn
```

You cannot directly access the ATM's internal system.

You interact through the provided interface.

This is Encapsulation.

Why Use Encapsulation?

- Protect data from unauthorized access.
 - Improve security.
 - Control how data is modified.
 - Make code easier to maintain.
 - Hide implementation details.
-

Data Hiding in Python

Python does not have strict access modifiers like some other languages, but it follows naming conventions.

Python provides three access levels:

1. Public Members
 2. Protected Members
 3. Private Members
-

1. Public Members

Public members can be accessed from anywhere.

Example

```
class Student:
    def __init__(self):
        self.name = "Alice"
```

```
s = Student()
```

```
print(s.name)
```

Output

```
Alice
```

2. Protected Members

Protected members are intended to be used within the class and its subclasses.

Protected members start with a single underscore (_).

Syntax

```
self._variable
```

Example

```
class Student:
    def __init__(self):
        self._name = "Alice"

s = Student()

print(s._name)
```

Output

```
Alice
```

Note

Protected members can still be accessed outside the class.

```
print(s._name)
```

Python treats this as a convention:

"Please don't access this directly."

3. Private Members

Private members cannot be accessed directly outside the class.

Private members begin with double underscores (__).

Syntax

```
self.__variable
```

Example

```
class Student:
    def __init__(self):
        self.__name = "Alice"

s = Student()

print(s.__name)
```

Output

AttributeError

Error:

AttributeError: 'Student' object has no attribute '__name'

Name Mangling

Python internally changes private variable names.

Example:

```
class Student:
    def __init__(self):
        self.__name = "Alice"

s = Student()

print(s._Student__name)
```

Output

Alice

Python converts:

`__name`

to

`_Student__name`

This process is called **Name Mangling**.

Encapsulation Using Getter and Setter Methods

Getter methods retrieve values.

Setter methods modify values.

Example

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance

    def get_balance(self):
        return self.__balance

    def set_balance(self, amount):
        if amount >= 0:
            self.__balance = amount
        else:
            print("Invalid amount")

account = BankAccount(1000)

print(account.get_balance())

account.set_balance(1500)

print(account.get_balance())
```

Output

```
1000
1500
```

Why Use Getters and Setters?

Suppose balance is public.

```
account.balance = -5000
```

Invalid data can be stored.

Using setters:

```
account.set_balance(-5000)
```

Output:

```
Invalid amount
```

Thus, data remains safe.

Encapsulation Using Properties

Python provides `property()` and decorators.

This is the recommended approach.

Example Using `@property`

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance

    @property
    def balance(self):
        return self.__balance

    @balance.setter
    def balance(self, amount):
        if amount >= 0:
            self.__balance = amount
        else:
            print("Balance cannot be negative")

account = BankAccount(1000)

print(account.balance)

account.balance = 2000

print(account.balance)
```

Output

```
1000
2000
```

Read-Only Property

A property with only a getter.

Example

```
class Person:
    def __init__(self, age):
        self.__age = age

    @property
    def age(self):
        return self.__age

p = Person(25)

print(p.age)
```

Output

25

Trying to modify:

```
p.age = 30
```

Output:

```
AttributeError
```

Write-Only Property

A property with only a setter.

Example

```
class Secret:
    def __init__(self):
        self.__password = ""

    @property
    def password(self):
        raise AttributeError("Password cannot be read")

    @password.setter
    def password(self, value):
        self.__password = value

s = Secret()

s.password = "abc123"
```

Complete Encapsulation Example

```
class Employee:
    def __init__(self, salary):
        self.__salary = salary

    @property
    def salary(self):
        return self.__salary

    @salary.setter
    def salary(self, amount):
        if amount > 0:
            self.__salary = amount
        else:
            print("Salary must be positive")

emp = Employee(30000)

print(emp.salary)
```

```
emp.salary = 40000  
  
print(emp.salary)  
  
emp.salary = -1000
```

Output

```
30000  
40000  
Salary must be positive
```

Advantages of Encapsulation

1. Data Security

Protects sensitive data.

2. Data Hiding

Internal implementation remains hidden.

3. Better Control

Validation can be applied before modifying data.

4. Flexibility

Implementation can change without affecting users.

5. Improved Maintenance

Makes large programs easier to manage.

Disadvantages of Encapsulation

- Slightly increases code complexity.
 - Requires additional methods or properties.
 - Can add a small amount of overhead.
-

Encapsulation vs Data Hiding

Encapsulation

Bundling data and methods together
Achieved using classes
Focuses on structure
Broader concept

Data Hiding

Restricting direct access to data
Achieved using private members
Focuses on security
Part of encapsulation

Encapsulation vs Abstraction

Encapsulation

Hides data
Focuses on "How data is protected"
Uses private variables and properties
Security-oriented

Abstraction

Hides implementation details
Focuses on "What an object does"
Uses abstract classes and methods
Design-oriented

Real-Life Example: Student Management System

```
class Student:
    def __init__(self, marks):
        self.__marks = marks

    @property
    def marks(self):
        return self.__marks

    @marks.setter
    def marks(self, value):
        if 0 <= value <= 100:
            self.__marks = value
        else:
            print("Marks must be between 0 and 100")

s = Student(85)

print(s.marks)

s.marks = 95
print(s.marks)

s.marks = 120
```

Output

```
85
95
Marks must be between 0 and 100
```

Summary

- **Encapsulation** means bundling data and methods into a class and restricting direct access to data.
- Python uses:
 - Public members → `name`
 - Protected members → `_name`
 - Private members → `__name`
- Private members use **Name Mangling**.
- Getters and Setters provide controlled access.
- `@property` is the Pythonic way to implement encapsulation.
- Encapsulation improves security, maintainability, and flexibility.

Formula to Remember

Encapsulation = Data + Methods + Controlled Access

Easy Memory Trick

Class → Stores Data
Private Variables → Protect Data
Getter/Setter or Properties → Control Access
Encapsulation → Safe and Organized Code

Link : CHATGPT

Abstraction in OOP (Python) – Detailed Explanation

Abstraction is one of the four main pillars of Object-Oriented Programming (OOP).

The four pillars are:

1. Encapsulation
 2. Inheritance
 3. Polymorphism
 4. Abstraction
-

What is Abstraction?

Abstraction means hiding unnecessary implementation details and showing only the essential features to the user.

In simple words:

Abstraction focuses on **what an object does**, not **how it does it**.

Real-Life Example

ATM Machine

```
Insert Card  
Enter PIN  
Withdraw Money
```

You know how to use the ATM.

But you do not know:

- How PIN verification works
- How the bank server processes requests
- How cash is counted internally

These details are hidden.

This is **Abstraction**.

Another Example

Car

Start Car
Accelerate
Brake

You use these features without knowing:

- Engine combustion process
- Fuel injection mechanism
- Internal wiring system

The complexity is hidden.

Why Use Abstraction?

- Hides complex implementation.
 - Improves security.
 - Makes code easier to use.
 - Reduces code complexity.
 - Improves maintainability.
-

Abstraction in Python

Python provides abstraction through the **ABC (Abstract Base Class)** module.

ABC stands for:

Abstract Base Class

It is available in the built-in module:

```
from abc import ABC, abstractmethod
```

Abstract Class

An abstract class is a class that contains one or more abstract methods.

An abstract class:

- Cannot be instantiated.
 - Acts as a blueprint for child classes.
 - Can contain abstract and normal methods.
-

Abstract Method

An abstract method:

- Has only a declaration.
 - Does not have implementation.
 - Must be implemented by child classes.
-

Syntax

```
from abc import ABC, abstractmethod

class Parent(ABC):

    @abstractmethod
    def method_name(self):
        pass
```

Basic Example

```
from abc import ABC, abstractmethod

class Animal(ABC):

    @abstractmethod
    def sound(self):
        pass

class Dog(Animal):

    def sound(self):
        print("Dog barks")

d = Dog()
d.sound()
```

Output

Dog barks

Why Can't We Create an Object of an Abstract Class?

```
from abc import ABC, abstractmethod

class Animal(ABC):

    @abstractmethod
    def sound(self):
        pass

a = Animal()
```

Output

```
TypeError:
Can't instantiate abstract class Animal
```

Because abstract classes are incomplete.

Abstract Class with Multiple Abstract Methods

```
from abc import ABC, abstractmethod

class Shape(ABC):

    @abstractmethod
    def area(self):
        pass

    @abstractmethod
    def perimeter(self):
        pass
```

Implementing the Methods

```
from abc import ABC, abstractmethod

class Shape(ABC):

    @abstractmethod
    def area(self):
        pass

    @abstractmethod
    def perimeter(self):
        pass
```

```
class Rectangle(Shape):

    def __init__(self, length, width):
        self.length = length
        self.width = width

    def area(self):
        return self.length * self.width

    def perimeter(self):
        return 2 * (self.length + self.width)

r = Rectangle(10, 5)

print("Area:", r.area())
print("Perimeter:", r.perimeter())
```

Output

```
Area: 50
Perimeter: 30
```

Child Class Must Implement All Abstract Methods

Example:

```
from abc import ABC, abstractmethod

class Shape(ABC):

    @abstractmethod
    def area(self):
        pass

    @abstractmethod
    def perimeter(self):
        pass

class Rectangle(Shape):

    def area(self):
        return 100
```

Output

```
TypeError
```

Reason:

```
perimeter() is not implemented.
```

Abstract Class Can Have Normal Methods

```
from abc import ABC, abstractmethod

class Vehicle(ABC):

    @abstractmethod
    def start(self):
        pass

    def stop(self):
        print("Vehicle Stopped")
```

Child Class

```
class Car(Vehicle):

    def start(self):
        print("Car Started")

c = Car()

c.start()
c.stop()
```

Output

```
Car Started
Vehicle Stopped
```

Abstract Class Can Have Constructor

```
from abc import ABC, abstractmethod

class Person(ABC):

    def __init__(self, name):
        self.name = name

    @abstractmethod
    def show(self):
        pass
```

Child Class

```
class Student(Person):

    def show(self):
        print("Name:", self.name)
```

```
s = Student("John")
s.show()
```

Output

Name: John

Real-Life Example: Payment System

```
from abc import ABC, abstractmethod

class Payment(ABC):

    @abstractmethod
    def pay(self, amount):
        pass
```

Child Classes

```
class UPI(Payment):

    def pay(self, amount):
        print("Paid", amount, "through UPI")

class CreditCard(Payment):

    def pay(self, amount):
        print("Paid", amount, "through Credit Card")

class Cash(Payment):

    def pay(self, amount):
        print("Paid", amount, "through Cash")
```

Using Objects

```
payments = [
    UPI(),
    CreditCard(),
    Cash()
]

for payment in payments:
    payment.pay(1000)
```

Output

```
Paid 1000 through UPI
Paid 1000 through Credit Card
Paid 1000 through Cash
```

Abstraction Using Interface-Like Structure

Python does not have interfaces like Java.

Abstract classes are used instead.

```
from abc import ABC, abstractmethod

class Printer(ABC):

    @abstractmethod
    def print_document(self):
        pass
```

Implementing Class

```
class LaserPrinter(Printer):

    def print_document(self):
        print("Printing Document")
```

Advantages of Abstraction

1. Hides Complexity

Users see only important features.

2. Improves Security

Internal implementation remains hidden.

3. Easy Maintenance

Implementation can change without affecting users.

4. Better Reusability

Common structure can be shared.

5. Improves Flexibility

Multiple implementations can exist.

Disadvantages of Abstraction

- Increases design complexity.
- Requires additional classes.
- More planning is needed.

Abstraction vs Encapsulation

Abstraction	Encapsulation
Hides implementation details	Hides data
Focuses on what to do	Focuses on how data is protected
Achieved using abstract classes	Achieved using private variables
Design-oriented	Security-oriented

Abstraction vs Inheritance

Abstraction	Inheritance
Defines common blueprint	Reuses existing code
Uses abstract methods	Uses parent-child relationship
Focuses on behavior	Focuses on structure

Abstraction vs Polymorphism

Abstraction	Polymorphism
Defines methods	Implements methods differently
Blueprint	Multiple behaviors
Uses abstract classes	Uses overriding

Complete Example

```
from abc import ABC, abstractmethod

class Employee(ABC):

    @abstractmethod
    def salary(self):
        pass

class Manager(Employee):

    def salary(self):
        print("Salary: 80000")
```

```
class Developer(Employee):  
    def salary(self):  
        print("Salary: 50000")  
  
employees = [  
    Manager(),  
    Developer()  
]  
  
for emp in employees:  
    emp.salary()
```

Output

```
Salary: 80000  
Salary: 50000
```

Summary

Abstraction

- Hides implementation details.
- Shows only essential features.
- Achieved using ABC module.
- Uses @abstractmethod.
- Abstract classes cannot be instantiated.
- Child classes must implement abstract methods.

Important Terms

Term	Meaning
ABC	Abstract Base Class
Abstract Class	Class containing abstract methods
Abstract Method	Method declared but not implemented
@abstractmethod	Decorator used to create abstract methods

Formula to Remember

Abstraction = Hide Implementation + Show Essential Features

Easy Memory Trick

Encapsulation → Hide Data
Inheritance → Reuse Code
Polymorphism → One Interface, Many Forms
Abstraction → Hide Complexity

OOP Pillars at a Glance

Encapsulation → Protect Data

Inheritance → Reuse Code

Polymorphism → Multiple Behaviors

Abstraction → Hide Implementation

Link : CHATGPT

Magic Methods in Python (Special Methods) – Detailed Explanation

What are Magic Methods?

Magic Methods are special methods in Python that start and end with double underscores (`__`).

They are also called:

- Special Methods
- Dunder Methods (Double Under Methods)

Examples:

```
__init__()  
__str__()  
__len__()  
__add__()  
__eq__()
```

These methods allow us to define how objects behave with built-in functions, operators, and language features.

Why Use Magic Methods?

Magic methods allow custom objects to:

- Use operators (+, -, *)
 - Work with `print()`
 - Work with `len()`
 - Compare objects
 - Iterate through objects
 - Behave like built-in data types
-

Example Without Magic Method

```
class Student:  
    pass  
  
s = Student()  
  
print(s)
```

Output

```
<__main__.Student object at 0x000001F2A8B5D6A0>
```

Not user-friendly.

Example With `__str__()`

```
class Student:
    def __str__(self):
        return "Student Object"
```

```
s = Student()
```

```
print(s)
```

Output

```
Student Object
```

Categories of Magic Methods

1. Constructor Methods
 2. String Representation Methods
 3. Comparison Methods
 4. Arithmetic Methods
 5. Length and Container Methods
 6. Iteration Methods
 7. Callable Methods
 8. Context Manager Methods
-

1. Constructor Magic Methods

`__init__()`

Called automatically when an object is created.

```
class Student:
    def __init__(self, name):
        self.name = name
```

```
s = Student("John")
```

```
print(s.name)
```

Output

John

`__del__()`

Called when an object is destroyed.

```
class Test:

    def __del__(self):
        print("Object Destroyed")

t = Test()

del t
```

Output

Object Destroyed

2. String Representation Methods

`__str__()`

Used by `print()`.

```
class Student:

    def __str__(self):
        return "Student Class"

s = Student()

print(s)
```

Output

Student Class

`__repr__()`

Used for developer-friendly representation.

```
class Student:

    def __repr__(self):
        return "Student() "

s = Student()
```

```
print(repr(s))
```

Output

```
Student()
```

Difference Between `__str__()` and `__repr__()`

<code>__str__()</code>	<code>__repr__()</code>
User-friendly output	Developer-friendly output
Used by <code>print()</code>	Used by <code>repr()</code>
Readable format	Detailed format

3. Arithmetic Magic Methods

These methods overload operators.

`__add__()`

Operator +

```
class Number:
    def __init__(self, value):
        self.value = value

    def __add__(self, other):
        return self.value + other.value

n1 = Number(10)
n2 = Number(20)

print(n1 + n2)
```

Output

```
30
```

`__sub__()`

Operator -

```
class Number:
    def __init__(self, value):
```

```
        self.value = value

    def __sub__(self, other):
        return self.value - other.value

n1 = Number(20)
n2 = Number(5)

print(n1 - n2)
```

Output

15

`__mul__()`

Operator *

```
class Number:

    def __init__(self, value):
        self.value = value

    def __mul__(self, other):
        return self.value * other.value

n1 = Number(5)
n2 = Number(4)

print(n1 * n2)
```

Output

20

`__truediv__()`

Operator /

```
class Number:

    def __init__(self, value):
        self.value = value

    def __truediv__(self, other):
        return self.value / other.value

n1 = Number(20)
n2 = Number(5)

print(n1 / n2)
```

Output

4. Comparison Magic Methods

`__eq__()`

Equal (==)

```
class Student:

    def __init__(self, marks):
        self.marks = marks

    def __eq__(self, other):
        return self.marks == other.marks

s1 = Student(90)
s2 = Student(90)

print(s1 == s2)
```

Output

True

`__lt__()`

Less Than (<)

```
class Student:

    def __init__(self, marks):
        self.marks = marks

    def __lt__(self, other):
        return self.marks < other.marks

s1 = Student(80)
s2 = Student(90)

print(s1 < s2)
```

Output

True

Other Comparison Methods

Method Operator

```
__eq__() ==  
__ne__() !=  
__lt__() <  
__le__() <=  
__gt__() >  
__ge__() >=
```

5. Length and Container Methods

`__len__()`

Used by `len()`.

```
class MyList:  
    def __len__(self):  
        return 5  
  
obj = MyList()  
print(len(obj))
```

Output

5

`__getitem__()`

Used for indexing.

```
class Numbers:  
    def __init__(self):  
        self.data = [10, 20, 30]  
  
    def __getitem__(self, index):  
        return self.data[index]  
  
n = Numbers()  
print(n[1])
```

Output

20

`__setitem__()`

Used for assignment.

```
class Numbers:

    def __init__(self):
        self.data = [10, 20, 30]

    def __setitem__(self, index, value):
        self.data[index] = value

n = Numbers()

n[1] = 50

print(n.data)
```

Output

```
[10, 50, 30]
```

6. Iteration Magic Methods

`__iter__()`

Returns iterator object.

`__next__()`

Returns next value.

Example

```
class Counter:

    def __init__(self):
        self.count = 1

    def __iter__(self):
        return self

    def __next__(self):
        if self.count <= 5:
            value = self.count
            self.count += 1
            return value
        raise StopIteration

c = Counter()

for num in c:
    print(num)
```


Output

```
1
2
3
4
5
```

7. Callable Objects

`__call__()`

Makes an object callable like a function.

```
class Greeting:

    def __call__(self):
        print("Hello")

g = Greeting()

g()
```

Output

```
Hello
```

8. Context Manager Methods

Used with `with` statement.

`__enter__()`

Executed when entering the block.

`__exit__()`

Executed when leaving the block.

Example

```
class FileManager:

    def __enter__(self):
        print("File Opened")
```

```
        return self

    def __exit__(self, exc_type, exc_value, traceback):
        print("File Closed")

with FileManager():
    print("Working...")
```

Output

```
File Opened
Working...
File Closed
```

Most Commonly Used Magic Methods

Magic Method	Purpose
<code>__init__()</code>	Constructor
<code>__str__()</code>	String representation
<code>__repr__()</code>	Official representation
<code>__len__()</code>	Length
<code>__add__()</code>	Addition
<code>__sub__()</code>	Subtraction
<code>__mul__()</code>	Multiplication
<code>__truediv__()</code>	Division
<code>__eq__()</code>	Equality
<code>__lt__()</code>	Less than
<code>__iter__()</code>	Iterator
<code>__next__()</code>	Next value
<code>__call__()</code>	Callable object
<code>__enter__()</code>	Enter context
<code>__exit__()</code>	Exit context

Real-Life Example: Bank Account

```
class BankAccount:

    def __init__(self, balance):
        self.balance = balance

    def __str__(self):
        return f"Balance: {self.balance}"

    def __add__(self, other):
        return BankAccount(self.balance + other.balance)
```

```
a1 = BankAccount(1000)
a2 = BankAccount(2000)

a3 = a1 + a2

print(a3)
```

Output

Balance: 3000

Advantages of Magic Methods

- Makes objects behave like built-in types.
 - Supports operator overloading.
 - Improves code readability.
 - Provides integration with Python's built-in functions.
 - Enables custom object behavior.
-

Summary

Magic Methods are special methods surrounded by double underscores (`__`).

Examples:

```
__init__()
__str__()
__repr__()
__len__()
__add__()
__eq__()
__iter__()
__next__()
__call__()
```

They allow custom objects to:

- Use operators
- Work with built-in functions
- Support iteration
- Act like functions
- Work with context managers

Formula to Remember

Magic Methods = Special Methods that Define Object Behavior

Easy Memory Trick

<code>__init__</code>	→ Create Object
<code>__str__</code>	→ Print Object
<code>__len__</code>	→ Length
<code>__add__</code>	→ +
<code>__eq__</code>	→ ==
<code>__call__</code>	→ Function Call
<code>__iter__</code>	→ Loop Start
<code>__next__</code>	→ Next Item

Link : [CHATGPT](#)

Link : [ALL](#)